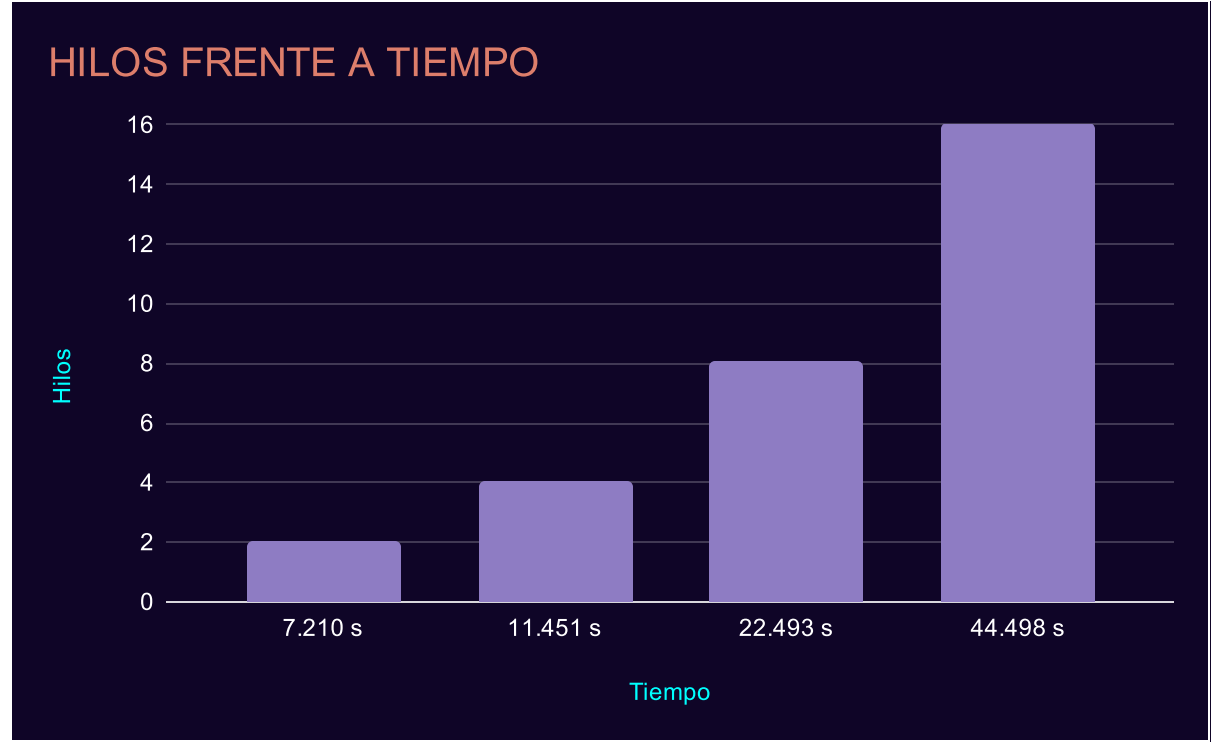


Hilos	Puntos	#	pi calculado	Tiempo
2	100000000		3.141.659	7.210 s
4	100000000		3.141.533	11.451 s
8	100000000		3.141.743	22.493 s
16	100000000		3.141.892	44.498 s



Explicación del Algoritmo Implementado (Test-and-Set)

El algoritmo **Test-and-Set (TAS)** es un mecanismo de sincronización por hardware que lee y modifica el estado de una variable de cerrojo de manera atómica (en un solo ciclo indivisible).

- **Entrada a la sección crítica:** Cada hilo ejecuta un bucle de espera activa (*spinlock*) evaluando `__atomic_test_and_set`. Si el cerrojo vale 0, la función lo cambia a 1 y le permite la entrada. Si vale 1, el hilo se queda atascado girando en el bucle consumiendo ciclos de CPU hasta que se libere.
- **Exclusión mutua:** Solo un hilo a la vez puede ejecutar la instrucción `puntosDentro++`.
- **Salida de la sección crítica:** Al finalizar la modificación, el hilo ejecuta `__atomic_clear` para restablecer el cerrojo a 0 y permitir la entrada de otro hilo.

Conclusión de Rendimiento

El mejor rendimiento se obtuvo con **2 hilos**, completando la ejecución en **7.210 segundos**.

A medida que se incrementó la cantidad de hilos (de 2 a 16), el tiempo de ejecución **empeoró drásticamente**, pasando de 7.21s hasta 44.498s. Esto ocurrió por las siguientes razones:

1. **Alta Contención en la Sección Crítica:** Como la instrucción `puntosDentro++` se ejecuta millones de veces dentro del bucle de cada hilo, la probabilidad de que varios hilos intenten adquirir el cerrojo simultáneamente es altísima.
2. **Degradación por Espera Activa (*Spinning*):** En lugar de suspender los hilos en espera, Test-and-Set los mantiene ejecutando bucles infinitos. Esto provocó que el tiempo de CPU en modo usuario aumentara exorbitantemente (de 14.16s a 151.99s), desperdiciando procesamiento en la negociación del cerrojo en lugar de hacer trabajo útil.
3. **Saturación del Bus de Memoria:** Con 8 y 16 hilos, la ejecución repetida de instrucciones atómicas sobre la misma dirección de memoria provocó un cuello de

botella en el hardware, saturando la coherencia de caché y desacelerando el programa.